

Universidad Nacional de Colombia

Sede Manizales

Departamento de Ingeniería Eléctrica, Electrónica y Computación

Manual de Usuario y Guía de Despliegue: Implementación, Simulación y Síntesis del Microcontrolador FemtoUN basado en RISC-V

Esta guía práctica documenta el flujo completo desde el código HDL (Verilog) hasta la programación en FPGA y la validación por puerto serie UART, usando herramientas de código abierto y la plataforma Tang Primer 20K.

Autores: Laura Alarcón, Sebastián Orozco

Hardware objetivo: Tang Primer 20K (Gowin GW2A-18)

Índice

1. Introducción y Contexto	3
1.1. ¿Qué es el FemtoUN?	3
1.2. Arquitectura del sistema	3
1.3. Mapa de memoria	3
2. Instalación de Herramientas	4
2.1. Resumen de herramientas necesarias	4
2.2. Descarga de Gowin EDA	4
2.3. Instalación del driver USB en Windows	5
2.4. Instalación de WSL Ubuntu	5
2.5. Instalación de paquetes en WSL Ubuntu	5
3. Obtención del Proyecto (Clonación)	6
3.1. Clonar desde GitHub	6
4. Compilación de Firmware	6
4.1. Programa: envío de mensaje por UART	7
4.2. Proceso completo de compilación	7
5. Simulación RTL con Icarus Verilog	8
5.1. ¿Qué es la simulación RTL?	8
5.2. Simulación con BRAM interna	8
5.3. Simulación con SPI Flash externa	9
6. Visualización de Señales con GTKWave	9
6.1. Abrir el archivo VCD	9
6.2. Agregar señales al visor	9
6.3. Señales clave y su significado	10
6.4. Máquina de estados del FemtoRV32 Quark	10
6.5. Decodificación de instrucciones RV32I	11
7. Síntesis en Gowin EDA	12
7.1. Creación del proyecto	12
7.2. Archivo de constraints físicos (.cst)	17
7.3. Ejecución de síntesis y Place & Route	17
8. Reporte de Recursos	18
8.1. Resultados de síntesis	18
8.2. Interpretación	18
9. Programación de la FPGA y Prueba UART	18
9.1. Conexión de la board	18
9.2. Descarga del Bitstream con Gowin Programmer	18
9.3. Validación por UART	20
10. Resolución de Problemas Comunes	21

11. Resumen del Flujo Completo

21

1 Introducción y Contexto

Objetivo de aprendizaje

Al finalizar esta guía, el usuario será capaz de: clonar el entorno del sistema, configurar las herramientas de software locales, compilar firmware bare-metal para RISC-V, realizar la simulación lógica RTL del procesador, visualizar e interpretar señales digitales en GTKWave, sintetizar el diseño en Gowin EDA y transferirlo a la FPGA Tang Primer 20K con comunicación serial UART funcional.

1.1 ¿Qué es el FemtoUN?

El **FemtoUN** es un SoC (System-on-Chip) educativo basado en la arquitectura de hardware de código abierto **RISC-V (RV32I)**. Su núcleo lógico principal se deriva de la variante *Quark* del diseño **FemtoRV32** de Bruno Levy, adaptado por la Universidad Nacional de Colombia. El proyecto fue diseñado para:

- Ser fabricado en tecnología **SkyWater 130 nm** mediante la plataforma TinyTapout.
- Ejecutarse en la FPGA **Tang Primer 20K** para validación previa al silicio.
- Servir como plataforma educativa en diseño digital y arquitectura de computadores.
- Integrar un periférico UART funcional para comunicación serial con el computador.

1.2 Arquitectura del sistema

Cuadro 1: Módulos del sistema FemtoUN

Módulo	Archivo	Función
Núcleo RISC-V	femtorv32_quark.v	Procesador RV32I + RDCYCLES
SOC con BRAM	SOC_bram.v	Sistema completo con memoria interna
BRAM con hex	bram_hex.v	Memoria inicializada desde archivo .hex
Periférico UART	perip_uart.v	Interfaz UART mapeada en memoria
UART física	uart.v	TX/RX con oversampling $\times 16$
Top FPGA	top_tang_bram.v	Wrapper para Tang Primer 20K

1.3 Mapa de memoria

Cuadro 2: Mapa de memoria del FemtoUN

Dirección	Periférico	Tamaño
0x00000000	BRAM (programa + datos)	1 KB (256 palabras $\times$ 32 bits)
0x00400000	UART (TX/RX/Status)	Registros mapeados en memoria

2 Instalación de Herramientas

Para completar el flujo desde el HDL hasta el silicio/FPGA programado, se requieren herramientas en Windows y en la terminal Linux (WSL).

2.1 Resumen de herramientas necesarias

Cuadro 3: Herramientas requeridas y sus roles

Herramienta	Sistema	Uso	Fuente
Gowin EDA V1.9.12+	Windows	Síntesis FPGA	gowinsemi.com
WSL Ubuntu 24.04	Windows	Compilación/simulación	Microsoft Store
Icarus Verilog	WSL	Simulación RTL	<code>apt install</code>
GTKWave	WSL	Visualizar ondas VCD	<code>apt install</code>
GCC RISC-V	WSL	Compilar firmware	<code>apt install</code>
PuTTY	Windows	Monitor serial UART	putty.org
Gowin USB Driver V5	Windows	Programar FPGA	Con Gowin EDA

2.2 Descarga de Gowin EDA

La síntesis lógica, el mapeo tecnológico y la generación del Bitstream final (.fs) requieren el suite oficial de Gowin Semiconductor.

1. Diríjase al portal de descargas:
https://gowinsemi.com/en/support/download_eda/. Debe registrarse pedir la licencia que tardará más o menos un día en llegarle al correo institucional.
2. Descargue la versión **Educational Edition** para Windows.

Download "GOWIN® EDA"

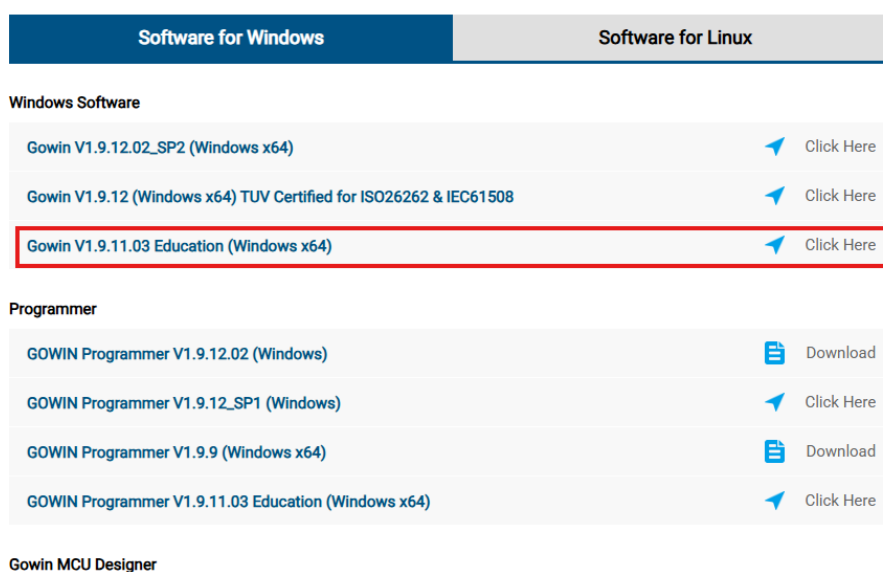


Figura 1: Portal oficial de descarga de Gowin EDA.

3. Complete el instalador incluyendo el componente **Gowin Programmer**.

2.3 Instalación del driver USB en Windows

1. Navega a:
`C:\Gowin\Gowin_V1.9.12.01_x64\Programmer\programmer\driver\`
2. Ejecuta como administrador:
`GowinUSBCableDriverV5_for_win7+.exe`
3. Desconecta y reconecta la board.
4. Verifica en el Administrador de dispositivos que aparecen **USB Serial Converter A y B**.

Advertencia

Si el USB Serial Converter B no aparece con COM asignado, instala el driver FT-DI VCP desde ftdichip.com/drivers/vcp-drivers/ (versión x64 para Windows 10/11). Luego en Administrador de dispositivos: clic derecho en USB Serial Converter B → Properties → Advanced → marcar **Load VCP**.

2.4 Instalación de WSL Ubuntu

1. Abra PowerShell en modo **Administrador**.
2. Instale Ubuntu con el siguiente comando:

Listing 1: Instalación de WSL Ubuntu 24.04

```
1 wsl --install -d Ubuntu-24.04
```

3. Reinicie el equipo cuando se solicite y configure usuario y contraseña para el entorno Linux.

2.5 Instalación de paquetes en WSL Ubuntu

Dentro de la terminal Ubuntu, instala todas las herramientas necesarias:

Listing 2: Instalación del toolchain open-source en WSL

```
1 # Actualizar repositorios
2 sudo apt update && sudo apt upgrade -y
3
4 # Simulador Verilog
5 sudo apt install iverilog -y
6
7 # Visualizador de ondas
8 sudo apt install gtkwave -y
9
10 # Make para automatizaci n
```

```
11 sudo apt install make -y
12
13 # Compilador cruzado RISC-V
14 sudo apt install gcc-riscv64-unknown-elf -y
15
16 # Programador de FPGA (opcional, alternativo a Gowin Programmer)
17 sudo apt install openfpgaloader -y
18
19 # Verificar instalaciones
20 iverilog -v
21 riscv64-unknown-elf-gcc --version
```

3 Obtención del Proyecto (Clonación)

3.1 Clonar desde GitHub

Todo el código validado está centralizado en el repositorio personal del proyecto. Para clonarlo en el entorno local:

Listing 3: Clonación de los repositorios del proyecto

```
1 # Directorio home del usuario
2 cd ~
3
4 # Repositorio simplificado del proyecto Femto (UNAL)
5 git clone https://github.com/LauraAlarcon14/FemtoUN.git
6 cd FemtoUN
7
8 # Repositorio principal (cicamargoba)
9 cd ~
10 git clone https://github.com/cicamargoba/VLSI
11 cd VLSI/femtoRV
12
13 # Repositorio de referencia (Bruno Levy, autor del FemtoRV32)
14 cd ~
15 git clone https://github.com/BrunoLevy/learn-fpga
```

Listing 4: Estructura del repositorio principal

```
1 ls ~/VLSI/femtoRV/
2 # SOC_flash.v    bench_quark_flash.v    cores/
3 # SOC_bram.v     bench_bram_TB.v       firmware/
4 # femto_TB.v     OpenLane/              synthesis/
5
6 ls ~/VLSI/femtoRV/cores/
7 # bram/  cpu/  uart/  mult/  spi_flash/  sim_spi_flash/
```

4 Compilación de Firmware

4.1 Programa: envío de mensaje por UART

Listing 5: hello.c – Programa bare-metal para UART

```

1  /* Mapa de registros UART en 0x00400000:
2  *   0x00400008 -> TX data   (escribir byte a transmitir)
3  *   0x00400010 -> Status    (bit 9 = tx_busy)
4  */
5  void main() {
6      volatile unsigned int *uart_tx   = (unsigned int*)0x00400008;
7      volatile unsigned int *uart_ctrl = (unsigned int*)0x00400010;
8
9      char *msg = "FemtoUN_RISC-V_corriendo!\n";
10
11     while (*msg) {
12         *uart_ctrl = 0x08;    /* tx_wr = 1 */
13         *uart_tx   = *msg++;  /* Enviar byte */
14         volatile int i;
15         for (i = 0; i < 5000; i++); /* Espera simple */
16     }
17     while (1); /* Loop infinito */
18 }

```

4.2 Proceso completo de compilación

Listing 6: Compilación de firmware para RV32I

```

1  # 1. Compilar con GCC para RISC-V
2  riscv64-unknown-elf-gcc \
3      -march=rv32i \           # Arquitectura RV32I
4      -mabi=ilp32 \           # ABI de 32 bits
5      -nostdlib \             # Sin librerías estándar
6      -nostartfiles \         # Sin startup de librerías
7      -T linker.ld \          # Linker script propio
8      -o firmware.elf \        # Salida ELF
9      start.S \               # Startup ASM
10     hello.c                   # Código C principal
11
12  # 2. Convertir ELF a binario plano
13  riscv64-unknown-elf-objcopy -O binary firmware.elf firmware.bin
14
15  # 3. Convertir binario a formato hex (32 bits por palabra) para
16     $readmemh
17  python3 << 'EOF'
18  import struct
19  with open('firmware.bin', 'rb') as f:
20      data = f.read()
21      while len(data) % 4:
22          data += b'\x00'
23      words = []
24      for i in range(0, len(data), 4):

```



```

24     word = struct.unpack('<I', data[i:i+4])[0]
25     words.append(f'{word:08x}')
26 while len(words) < 256:
27     words.append('00000013')    # NOP padding
28 with open('firmware.hex', 'w') as f:
29     f.write('\n'.join(words) + '\n')
30 print(f'Generadas {len(words)} palabras')
31 EOF
32
33 # 4. Verificar las primeras instrucciones
34 head -5 firmware.hex
35 # Debe mostrar:
36 # 00001137  <- LUI  x2    (carga SP)
37 # ffe10113  <- ADDI  x2,x2,-2
38 # 008000ef  <- JAL    (llama main)
39 # 0000006f  <- J  loop

```

5 Simulación RTL con Icarus Verilog

5.1 ¿Qué es la simulación RTL?

La simulación RTL (Register Transfer Level) permite verificar el comportamiento del diseño **sin necesitar hardware físico**. El simulador evalúa el código Verilog ciclo a ciclo de reloj y genera un archivo `.vcd` con las señales para análisis posterior.

5.2 Simulación con BRAM interna

El procesador ejecuta instrucciones directamente desde la memoria BRAM interna, inicializada con el archivo `firmware.hex`:

Listing 7: Compilar y simular con BRAM

```

1  cd ~/VLSI/femtoRV
2
3  # Compilar el testbench
4  iverilog -DBENCH -o sim_bram \
5      bench_bram_TB.v \
6      SOC_bram.v \
7      cores/cpu/femtorv32_quark.v \
8      cores/bram/bram_hex.v \
9      cores/uart/perip_uart.v \
10     cores/uart/uart.v \
11     -I cores/cpu
12
13 # Ejecutar la simulacion (genera bench.vcd)
14 vvp sim_bram
15 # Salida esperada:
16 # VCD info: dumpfile bench.vcd opened for output.
17 # bench_bram_TB.v:107: $finish called at 13198260 (1s)

```

5.3 Simulación con SPI Flash externa

Listing 8: Compilar y simular con SPI Flash

```
1 cd ~/VLSI/femtoRV
2
3 iverilog -DBENCH -o sim_quark \
4   bench_quark_flash.v \
5   OpenLane/src/femto.v \
6   OpenLane/src/MappedSPIRAM.v \
7   OpenLane/src/MappedSPIFlash.v \
8   cores/cpu/femtorv32_quark.v \
9   cores/bram/bram.v \
10  cores/uart/perip_uart.v \
11  cores/uart/uart.v \
12  cores/sim_spi_flash/spiflash.v \
13  cores/mult/perip_mult.v \
14  cores/mult/mult.v \
15  -I cores/cpu -I cores/bram -I cores/uart
16
17 vvp sim_quark
```

6 Visualización de Señales con GTKWave

6.1 Abrir el archivo VCD

Listing 9: Lanzamiento de GTKWave

```
1 # Desde WSL
2 gtkwave bench.vcd &
3
4 # O copiar a Windows y abrir desde ahi
5 cp bench.vcd /mnt/c/Users/TuUsuario/Desktop/bench.vcd
```

6.2 Agregar señales al visor

En GTKWave, en el panel izquierdo SST:

1. Expande bench → uut → CPU.
2. Selecciona las señales de interés.
3. Haz clic en **Append** para añadirlas al visor temporal.

6.3 Señales clave y su significado

Cuadro 4: Señales del FemtoRV32 Quark y su significado

Señal	Valor típico	Significado
clk	Oscilación continua	Reloj del sistema (27 MHz en FPGA, 25 MHz en sim)
resetn	0→1	Reset activo bajo: 0=reset, 1=corriendo
PC[23:0]	Incrementa de 4 en 4	Contador de programa, una instrucción por paso
instr[31:2]	Código instrucción	Instrucción RV32I ejecutándose (30 bits)
state[3:0]	1, 2, 4, 8	FSM: FETCH(1), WAIT(2), EXECUTE(4), WAIT_MEM(8)
mem_addr	Dirección de memoria	Dirección que el CPU está accediendo
mem_rdata	Dato leído	Instrucción o dato leído desde memoria
mem_rstrb	Pulso en 1	Señal de lectura activa
mem_wmask	0000 o 1111	Máscara de escritura en memoria
uart_tx	Pulsos NRZ	Datos saliendo por UART hacia el PC
spi_clk	Pulsos	Reloj de comunicación SPI con Flash
spi_mosi	Datos serie	Datos enviados al SPI Flash
spi_miso	Datos serie	Datos recibidos del SPI Flash

6.4 Máquina de estados del FemtoRV32 Quark

Cuadro 5: Estados de la FSM (codificación one-hot)

Estado	Valor	Acción	Señales activas
FETCH_INSTR	4'b0001	Solicita instrucción a memoria	mem_rstrb=1
WAIT_INSTR	4'b0010	Espera dato de memoria	Lee mem_rdata
EXECUTE	4'b0100	Ejecuta la instrucción	writeBack, aluWr
WAIT_ALU_OR_MEM	4'b1000	Espera operación lenta	Espera !mem_rbusy

6.5 Decodificación de instrucciones RV32I

Cuadro 6: Ejemplos de instrucciones RV32I observadas en simulación

Hex (32 bits)	Instrucción	Operación
00000013	NOP (ADDI x0,x0,0)	No hace nada
00001137	LUI x2, 0x1	Carga inmediato alto
ffe10113	ADDI x2,x2,-2	Ajusta stack pointer
008000ef	JAL x1, 8	Salta y enlaza (llamada a función)
0000006f	JAL x0, 0	Loop infinito

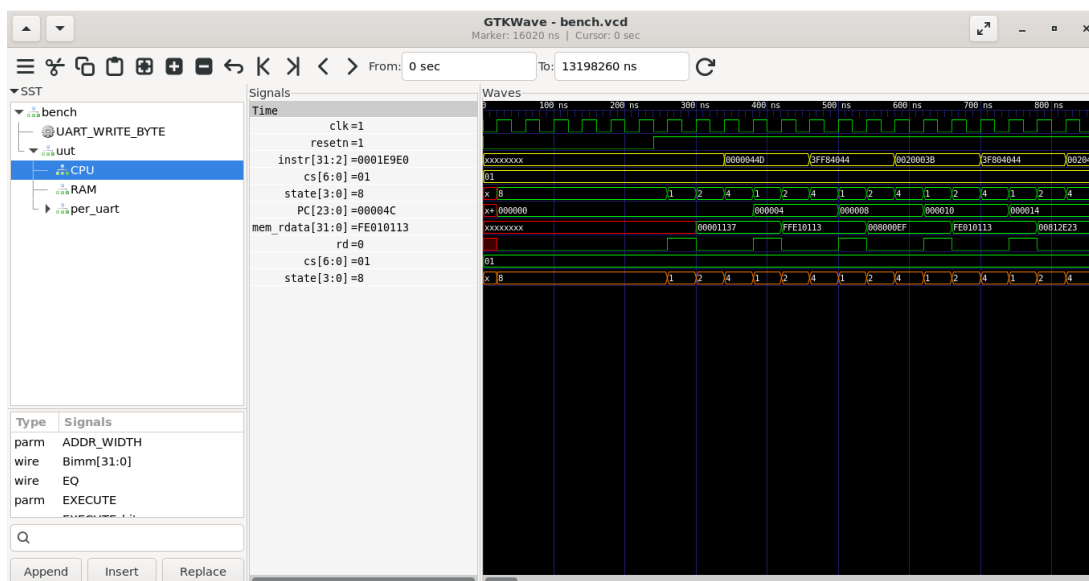


Figura 2: Simulación RTL del FemtoRV32 Quark con BRAM interna.

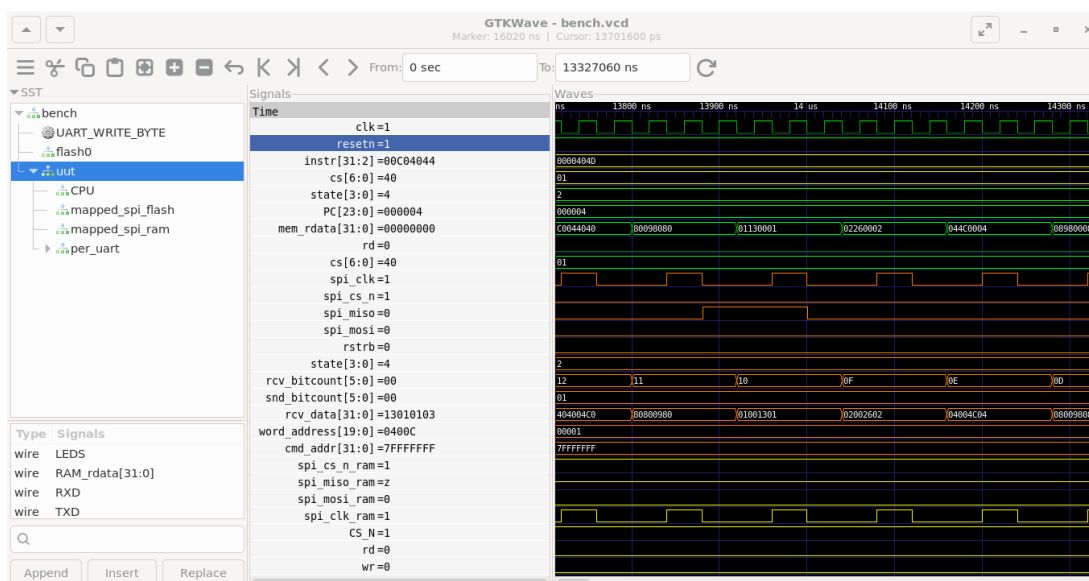


Figura 3: Simulación RTL del FemtoRV32 Quark con SPI Flash externa.

7 Síntesis en Gowin EDA

Una vez verificado el comportamiento en simulación, se procede a la síntesis física sobre la FPGA Tang Primer 20K.

7.1 Creación del proyecto

1. Abre Gowin EDA → **File** → **New Project**.
2. Nombre del proyecto: **FemtoUN_BRAM**.
3. En la selección de dispositivo, filtra por los siguientes valores:

Cuadro 7: Configuración del dispositivo en Gowin EDA

Parámetro	Valor
Series	GW2A
Device	GW2A-18 / GW2A-LV18
Package	PBGA256
Speed Grade	C8/I7
Part Number	GW2A-LV18PG256C8/I7

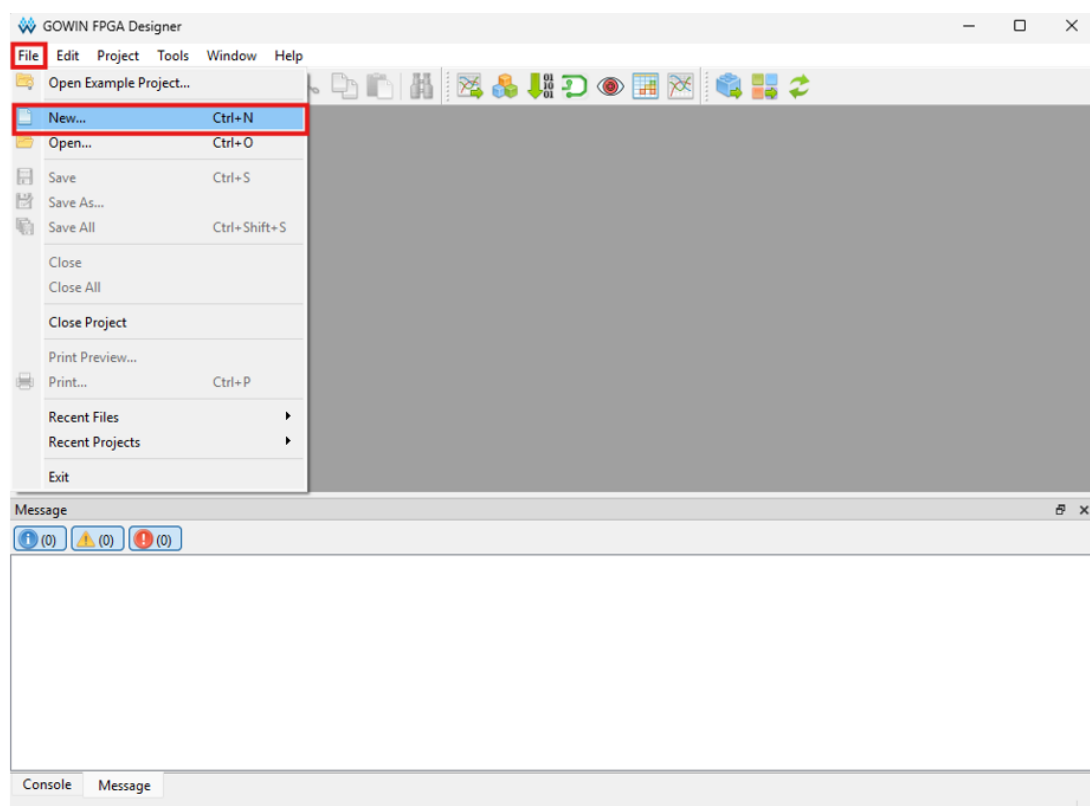


Figura 4: Nuevo proyecto.

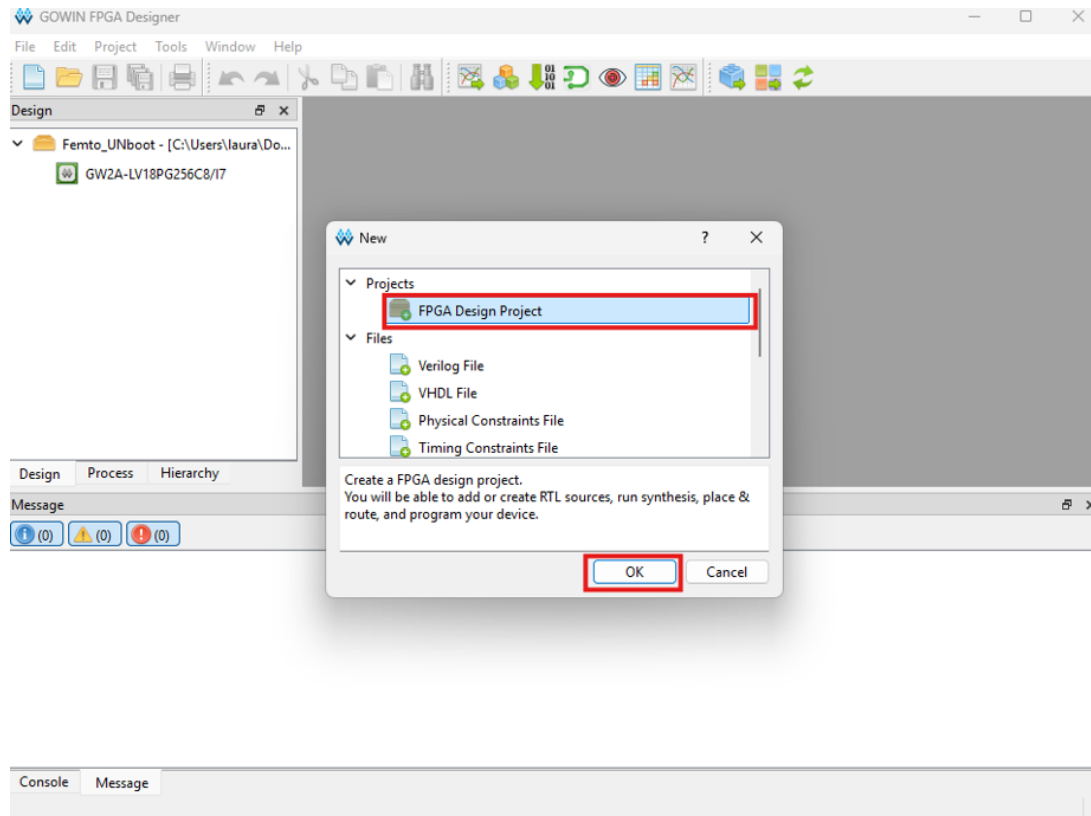


Figura 5: Seleccionamos FPGA Design Project y Ok.

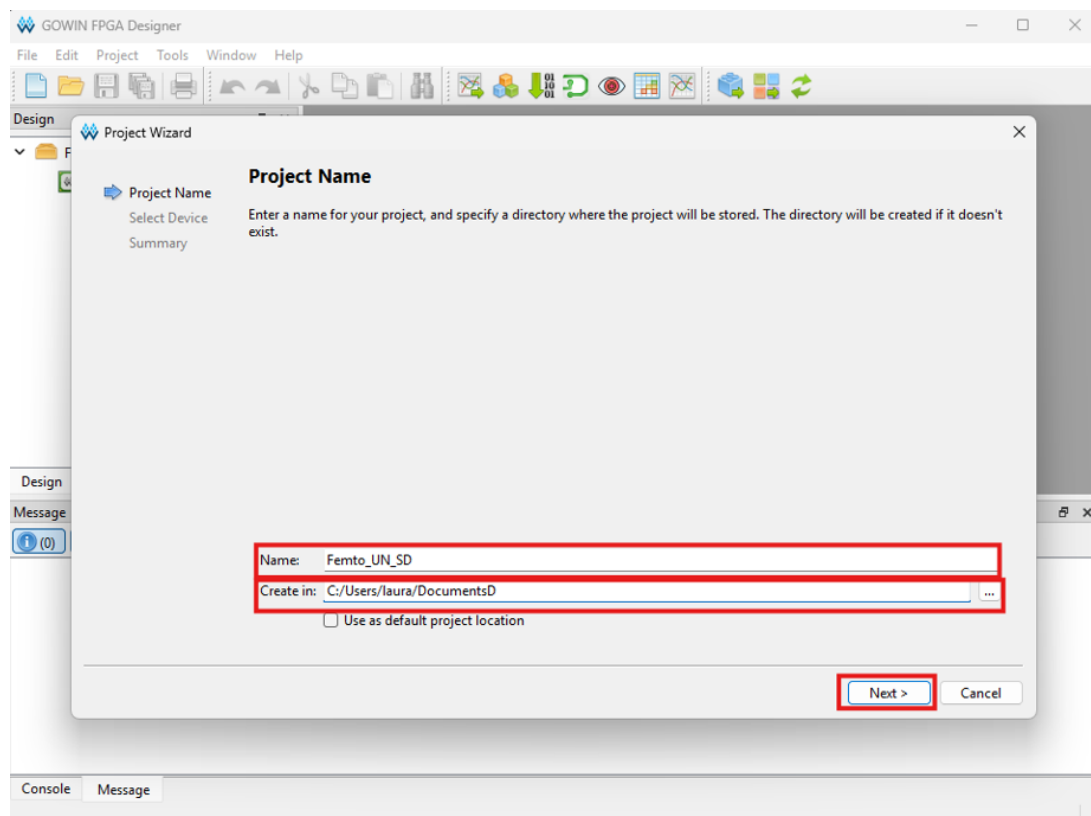


Figura 6: Colocamos un nombre a nuestro proyecto y nos fijamos en la ubicación del archivo y damos click en Next.

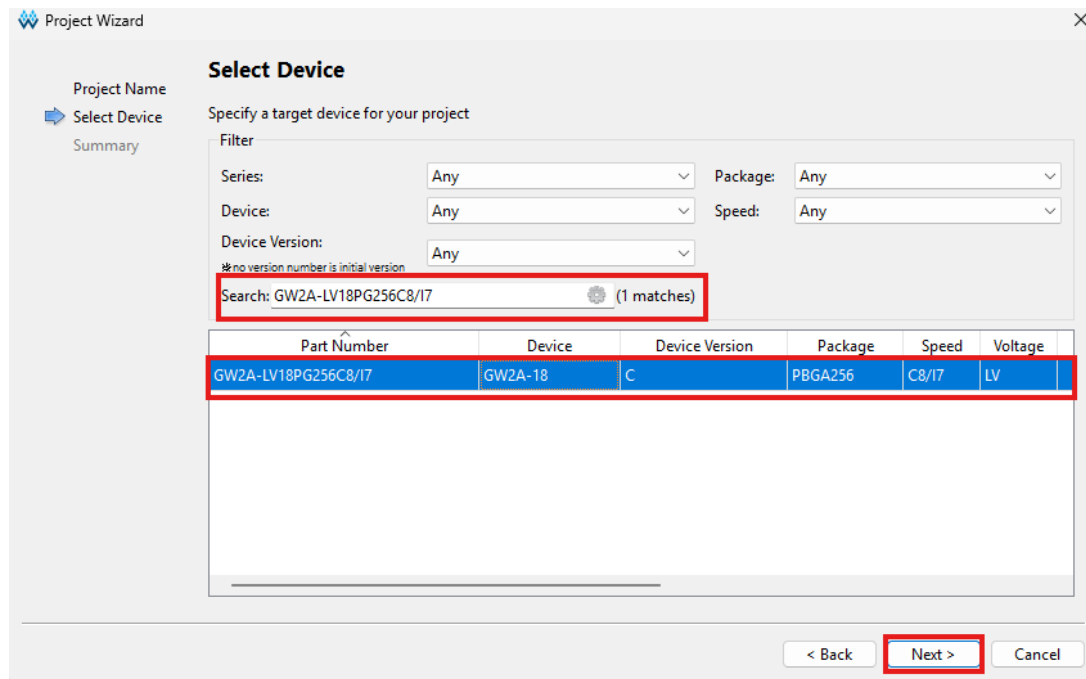


Figura 7: Configuración de la arquitectura física objetivo en Gowin EDA.

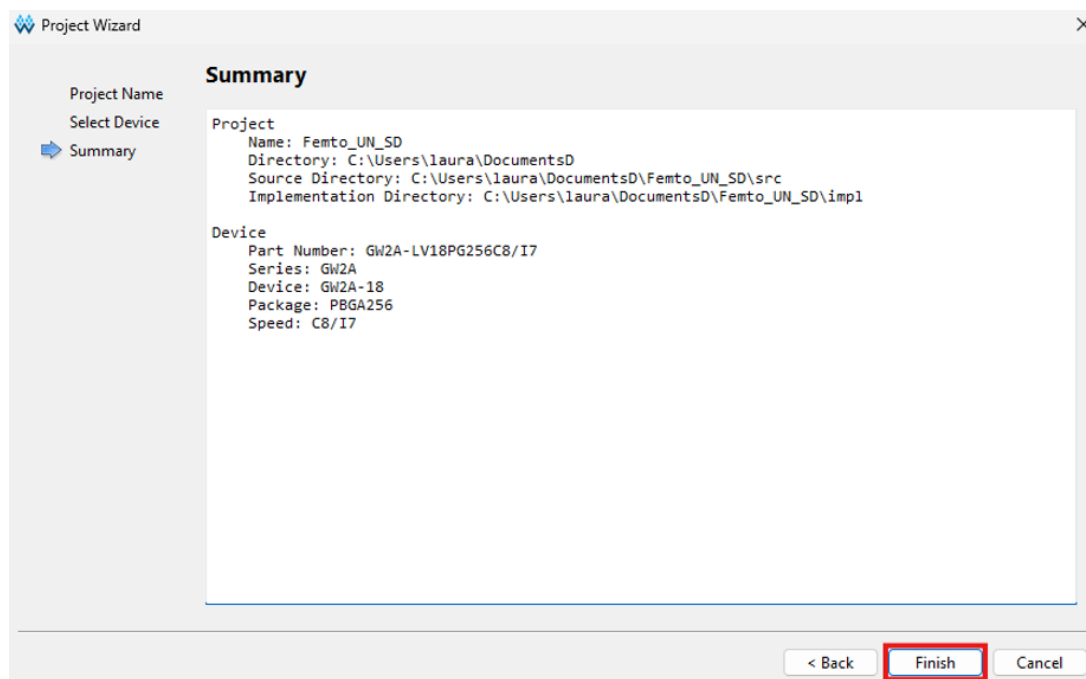


Figura 8: Finish y está listo el entorno para programar o copiar los archivos del repositorio guía.

Clic derecho en **Design** → **Add Files** y agrega:

- top_tang_bram.v → establecer como *Top Level*
- SOC_bram.v
- bram_hex.v

- femtorv32_quark.v
- perip_uart.v
- uart.v

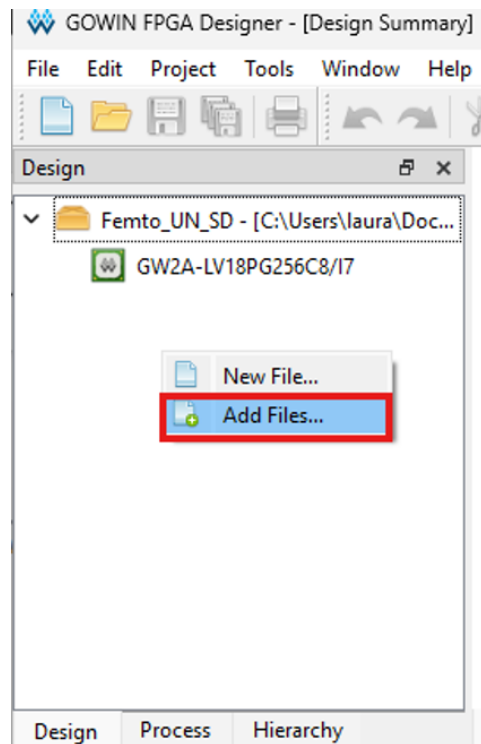


Figura 9: Click derecho y Add Files.

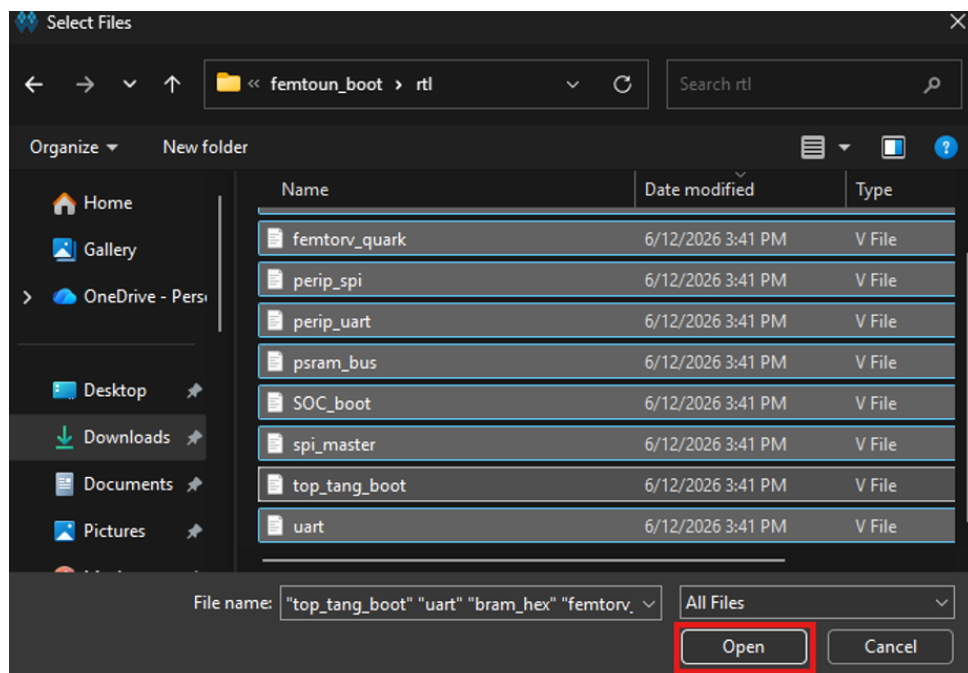


Figura 10: Archivos de la carpeta src para importar.

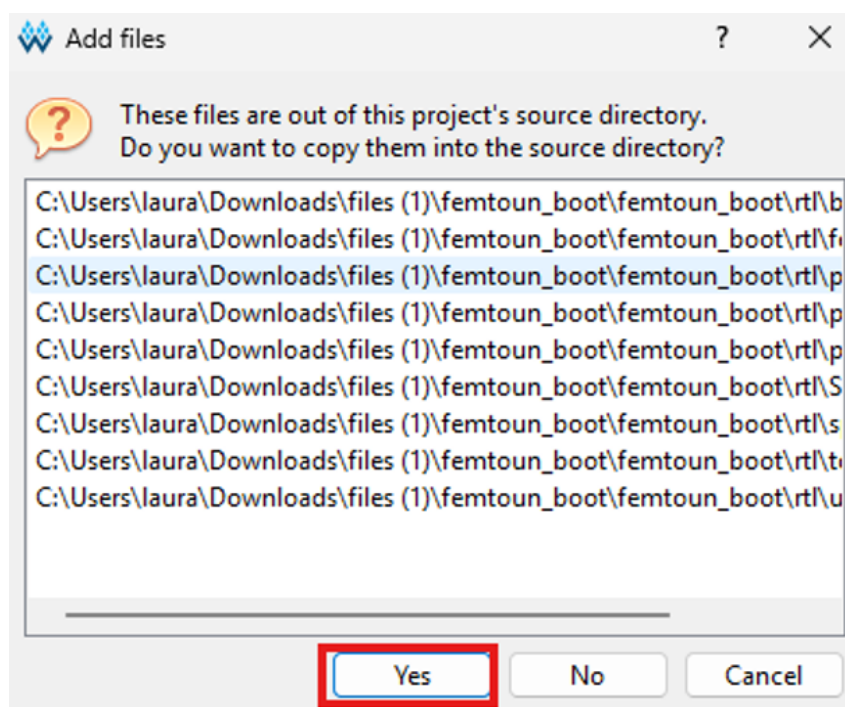


Figura 11: Advertencia que da el programa, dar click en Yes.

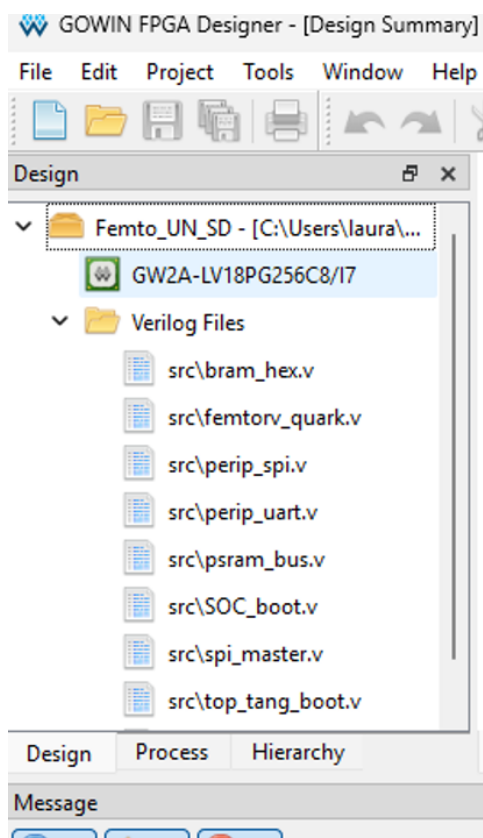


Figura 12: Estructura del proyecto con los módulos lógicos importados.

7.2 Archivo de constraints físicos (.cst)

Clic derecho en **Constrain** → **Add Files** → **Physical Constraints File**:

Listing 10: tang_bram.cst – Asignación de pines físicos para Tang Primer 20K

```
// Reloj principal 27 MHz
IO_LOC "clk_27mhz" H11;
IO_PORT "clk_27mhz" PULL_MODE=NONE IO_TYPE=LVC MOS33;

// Reset (boton S1, activo bajo)
IO_LOC "rst_n" E8;
IO_PORT "rst_n" PULL_MODE=UP IO_TYPE=LVC MOS33;

// UART TX (hacia PC)
IO_LOC "uart_tx" H6;
IO_PORT "uart_tx" PULL_MODE=NONE IO_TYPE=LVC MOS33;

// UART RX (desde PC)
IO_LOC "uart_rx" H5;
IO_PORT "uart_rx" PULL_MODE=UP IO_TYPE=LVC MOS33;

// LED indicador
IO_LOC "led" L16;
IO_PORT "led" PULL_MODE=NONE IO_TYPE=LVC MOS33;
```

Advertencia

Los pines SPI Flash (banco 5) operan a **1.8V (LVC MOS18)** y están conectados internamente. No asignarlos en el .cst con LVC MOS33: esto genera errores de síntesis (CT1136). Para verificar el pinout exacto, consultar el esquemático oficial en <https://github.com/sipeed/TangPrimer-20K>.

7.3 Ejecución de síntesis y Place & Route

En el panel **Process**, ejecutar en orden:

1. Doble clic en **Synthesize** → esperar indicador verde ().
2. Doble clic en **Place & Route** → esperar indicador verde ().
3. El Bitstream .fs se genera automáticamente en impl/pnr/.

Nota

Los warnings “*Latch inferred*” en femtorv32_quark.v son esperados y no impiden la síntesis. Para eliminarlos, usa la versión del repositorio cicamargoba/VLSI que declara `wire writeBack` en lugar de `reg writeBack`.

8 Reporte de Recursos

8.1 Resultados de síntesis

Tras el Place & Route, el reporte confirma que el diseño ocupa una fracción pequeña de la FPGA, validando su viabilidad para el tile TinyTapeout (4×2):

Cuadro 8: Recursos del FemtoUN (Quark + BRAM + UART) en GW2A-18

Recurso	Usado	Disponible	Utilización
Lógica total	1065	20736	6 %
LUT4	901	20736	4 %
ALU	164	—	—
Registros (FF)	314	15552	3 %
Latches	12	15552	<1 %
BSRAM	6	46	14 %
I/O Pins	5	207	3 %
PRIMARY Clock	3	8	38 %

8.2 Interpretación

- **901 LUTs:** El núcleo RV32I + UART ocupa menos del 5 % de la FPGA; hay amplio margen para agregar periféricos.
- **6 BRAMs:** La memoria de programa usa 6 de 46 bloques disponibles.
- **Viable para TinyTapeout:** El tile 4×2 ($\approx 668 \times 216 \mu\text{m}$) puede alojar este diseño completo.

9 Programación de la FPGA y Prueba UART

9.1 Conexión de la board

1. Conecta el cable USB-C al puerto **JTAG/UART** de la Tang Primer 20K.
2. Verifica en el Administrador de dispositivos que aparecen **USB Serial Converter A** (JTAG) y **B** (UART de datos).

9.2 Descarga del Bitstream con Gowin Programmer

1. En Gowin EDA, ve a **Tools** → **Programmer**.
2. Haz clic en **Scan Device** para detectar la FPGA.
3. Doble clic sobre el ícono del dispositivo y configura:
 - *Access Mode:* **SRAM Mode** (temporal, se borra al quitar energía).

- *FS File*: ruta al archivo .fs en impl/pnr/.
4. Presiona **Program/Configure** ().
 5. Resultado esperado en el log: **Info: Finished. Cost X second(s).**

Nota

Para programación permanente usa **Flash Program**. Requiere más tiempo y cuidado para no dañar la flash de la board. Para desarrollo iterativo, **SRAM Mode** es suficiente.

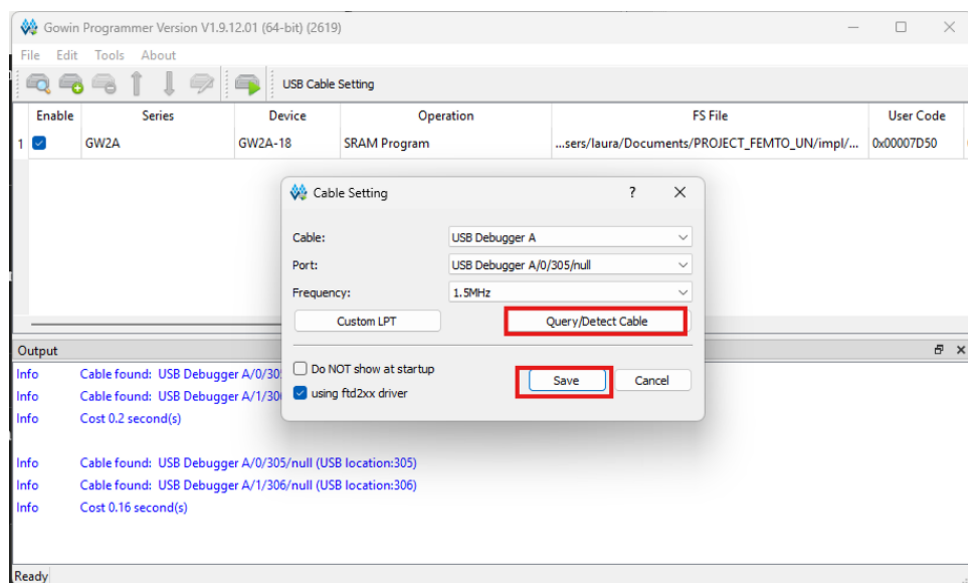


Figura 13: Primero comprobamos que el programa reconozca la placa y damos click en save.

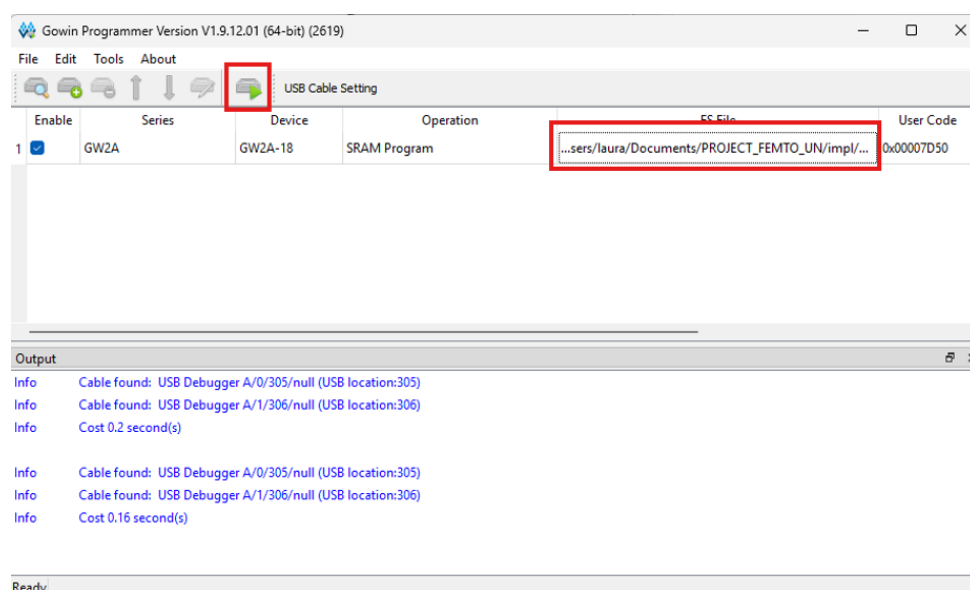


Figura 14: Transferencia del Bitstream final a la memoria SRAM interna, verificamos que esté seleccionado el archivo .fs y le damos en el recuadro de arriba para cargar el programa en la placa.

9.3 Validación por UART

El procesador FemtoUN comienza a ejecutar el firmware de inmediato en cuanto se completa la programación.

1. Abre **PuTTY** (o el monitor serie de Arduino).
2. Configuración: **Serial**, número de puerto **COM** del **canal B**, velocidad **115 200 baudios**, 8N1.
3. Presiona el botón de **Reset** físico en la board.
4. En pantalla aparece el mensaje enviado por el núcleo RISC-V.

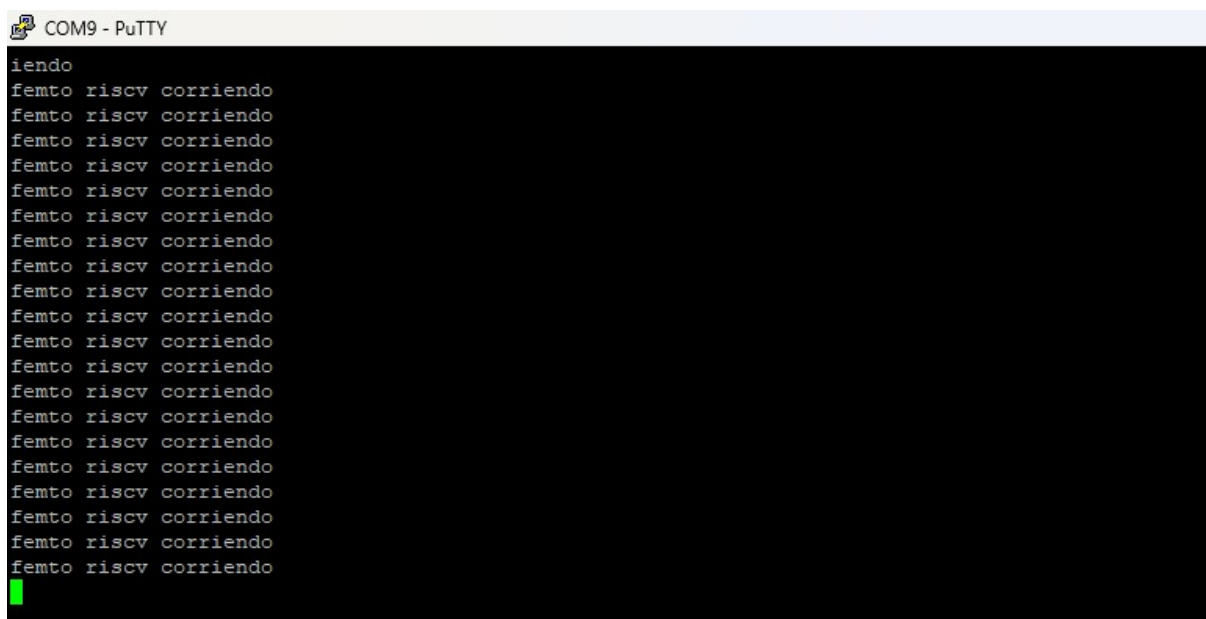


Figura 15: Consola de recepción verificando la funcionalidad del periférico UART.

10 Resolución de Problemas Comunes

Cuadro 9: Problemas frecuentes y soluciones

Problema	Causa	Solución
Error CT1136 banco 5	Pines SPI a 1.8V	Eliminar pines SPI del <code>.cst</code>
Pin no existe en el chip	Pinout incorrecto	Verificar esquemático de Si-speed
Pines duplicados en <code>.cst</code>	Ediciones superpuestas	Ctrl+A, Delete, reescribir
No aparece COM en Windows	Driver no instalado	Instalar GowinUSBCable-DriverV5
Cable USB no detectado	Cable solo de carga	Usar cable USB-C con datos
“Gowin Device not found”	Frecuencia JTAG alta	Bajar a 1 MHz en Cable Setting
Firmware no ejecuta	Dirección de enlace mal	Verificar linker script en <code>0x0</code>
PuTTY sin salida	COM incorrecto	Usar canal B (UART), no A (JTAG)
<code>config.mk</code> no encontrado	Falta en repositorio	Crear manualmente con <code>ARCH=rv32i</code>
Latches en síntesis	<code>reg writeBack</code>	Usar <code>wire writeBack</code> del repo VLSI

11 Resumen del Flujo Completo

Lista de chequeo: flujo completo FemtoUN

1. **Instalar herramientas:** WSL Ubuntu, iverilog, gtkwave, gcc-riscv64-unknown-elf, Gowin EDA.
2. **Clonar repositorios:** git clone de FemtoUN y VLSI.
3. **Compilar firmware:** GCC con `-march=rv32i`, linker en `0x0`, generar `firmware.hex`.
4. **Simular:** iverilog + vvp → genera `bench.vcd`.
5. **Analizar ondas:** GTKWave muestra PC, instrucciones y estados de la FSM.
6. **Sintetizar:** Gowin EDA → Synthesize → Place & Route → `.fs`.
7. **Verificar recursos:** Place & Route Report (6 % LUT, 14 % BSRAM).
8. **Programar FPGA:** Gowin Programmer → SRAM Program.
9. **Verificar UART:** PuTTY en COM canal B, 115 200 baudios.

Referencias

- [1] L. Alarcón, *Repositorio del Proyecto de Grado – Procesador FemtoUN*, <https://github.com/LauraAlarcon14/FemtoUN>, 2026.

- [2] C. Camargo, *Repositorio VLSI – UNAL*, <https://github.com/cicamargoba/VLSI>, 2024.
- [3] B. Levy, M. Koch, *FemtoRV32: A Minimalistic RISC-V CPU*, <https://github.com/BrunoLevy/learn-fpga>, 2020.
- [4] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Versión 2.2, 2017.
- [5] TinyTapeout, *TinyTapeout Documentation*, <https://tinytapeout.com>, 2023.
- [6] Sipeed, *Tang Primer 20K Schematic*, <https://github.com/sipeed/TangPrimer-20K>, 2022.
- [7] Gowin Semiconductor Corporation, *Gowin EDA Technical Support Portal*, https://gowinsemi.com/en/support/download_eda/, 2025.